

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CASE STUDY REPORT

UNIVERSITY HOSTEL AND ACCOMMODATION MANAGEMENT SYSTEM

Student Name: VAIBHAV BAKSHI

Register Number: 24BCE2052

Course Code: BCSE302P

Course Name: Database Systems Lab

Faculty Name: Baiju BV

Date of Submission: 29 / 10 / 2026

1. Introduction

The University Hostel and Accommodation Management System is a database-driven solution designed to automate and streamline the various activities involved in managing university hostels. The main objective of this system is to maintain accurate and centralized records of students, hostels, rooms, beds, allocations, fees, attendance, complaints, visitors and disciplinary incidents.

Traditionally, hostel administration is carried out manually using registers and spreadsheets, which is time-consuming, error-prone and often leads to data redundancy. This project overcomes these drawbacks by implementing a Relational Database Management System (RDBMS) using Oracle SQL to ensure efficient data storage, retrieval and management.

The system takes care of essential functions such as allotting rooms and beds to students, tracking fee charges and payments, monitoring student attendance, handling maintenance complaints, recording visitor entries and exits, and maintaining disciplinary records. To ensure data consistency and integrity, various business rules have been enforced through integrity constraints and PL/SQL triggers.

Furthermore, analytical views and SQL queries have been created to generate meaningful reports such as hostel occupancy, fee balances, student support risk and overall hostel performance. These reports assist the hostel administration in monitoring resources, identifying students who require attention and making better decisions.

Overall, this system aims to improve the efficiency of hostel management, eliminate manual work, enforce strict business rules and provide a reliable platform for effective administration of student accommodation.

2. Requirement Analysis

2.1 Functional Requirements

The following are the functional requirements that the University Hostel and Accommodation Management System must perform:

• Student Management – The system must store and maintain student details such as registration number, student name, department, email, phone number and emergency contact.
• Hostel Management – The system must maintain details of all hostels along with their names and locations.
• Room and Bed Management – The system must manage rooms under each hostel, their room types (AC or NON-AC), beds present in each room and track the service status of each bed (AVAILABLE or OUT_OF_SERVICE).
• Allocation Management – The system must allot beds to students, maintain current and past allocations, and preserve the complete allocation history of every student.
• Fee Management – The system must generate fee charges for each active allocation on a billing period basis, record payments made against these charges and track the outstanding and overdue amounts.
• Attendance Management – The system must record the daily attendance of students for their active allocation period and classify it as PRESENT, ABSENT or APPROVED_LEAVE.
• Complaint Management – The system must allow complaints to be raised by students, categorise them (PLUMBING, ELECTRICAL, FURNITURE, CLEANING or NETWORK), track their status (OPEN, IN_PROGRESS or RESOLVED) and record the date of resolution.
• Maintenance Expense Tracking – The system must record the expenses incurred in order to resolve complaints raised by students.
• Visitor Management – The system must record visitor details, their relationship with the resident, entry time and exit time for visitors visiting students currently residing in the hostel.

• Disciplinary Management – The system must maintain records of disciplinary incidents, the type of incident, its severity (LOW, MEDIUM or HIGH) and the action taken against it.
• Reporting and Monitoring – The system must generate analytical reports such as hostel occupancy, fee collection, student attendance, support risk analysis and overall hostel performance to assist the hostel administration in decision making.

2.2 Business Rules

The following business rules have been identified and enforced within the system:
• Student Uniqueness – Each student must have a unique registration number and a unique email ID.
• Hostel and Room Uniqueness – Each hostel must have a unique hostel name. A room is uniquely identified by a combination of (hostel_id, room_no).
• Bed Management – Each bed is uniquely identified by a combination of (room_id, bed_no). A bed can either be AVAILABLE or OUT_OF_SERVICE. An out of service bed cannot be allocated to any student.
• Allocation Rules – An allocation links exactly one student to exactly one bed for a specific time period. If an end date is specified, it must be greater than the start date. Both dates must be truncated to the calendar day.
• No Overlapping Allocations – A single student cannot have two active allocations at the same time. Similarly, a single bed cannot be allotted to two different students simultaneously.
• History Preservation – The student_id or bed_id of an existing allocation cannot be updated. To transfer a student to a different bed, the current allocation must be closed by adding an end date and a new allocation must be created.
• Bed Restrictions – A bed that has any allocation history cannot be relocated to a different room. Additionally, a bed with a current or future allocation cannot be marked as OUT_OF_SERVICE.
• Fee Rules – For each allocation, only one fee charge can be created per billing period. The charge amount must be greater than 0 and the due date must be on or after the charge date.
• Payment Rules – Payments can only be made against valid fee charges. The total amount of payments for a charge cannot exceed the charge amount. A payment cannot be made before its corresponding charge date.
• Charge Modification Rules – A fee charge cannot be updated if it would make its amount less than the total payments already made against it, or if the charge date is changed to a date after the earliest payment made for that charge.
• Attendance Rules – Attendance can only be recorded for dates that fall within the active allocation period of the student. Only one attendance record is allowed per (allocation_id, attendance_date). Attendance status must be either PRESENT, ABSENT or APPROVED_LEAVE. Approved leave is excluded from the attendance percentage calculation.
• Complaint Rules – Every complaint must be linked to a valid student and a valid room. Complaints must belong to a valid category and status. A complaint can only be marked as RESOLVED if a resolved date is provided, and the resolved date must be greater than or equal to the raised date. For OPEN or IN_PROGRESS complaints, the resolved date must be NULL.
• Maintenance Expense Rules – Every maintenance expense must be linked to a valid complaint and the expense amount must be greater than or equal to 0.
• Visitor Rules – A visitor visit must be linked to a valid allocation. The exit time, if provided, must be greater than or equal to the entry time.
• Disciplinary Rules – Every disciplinary incident must be linked to a valid allocation and must have a valid severity of LOW, MEDIUM or HIGH.
• Reporting Rules – All analytical reports are generated based on the values defined in the report_config table (window_start, as_of_date and occupancy_benchmark).

2.3 Entities, Attributes and Relationships

2.3.1 Entities and their Attributes

Entity	Attributes	Description
STUDENT	student_id (PK), registration_no (UNIQUE), student_name, department, email (UNIQUE), phone, emergency_contact	Stores personal and academic details of students.
HOSTEL	hostel_id (PK), hostel_name (UNIQUE), location	Stores details of university hostels.
ROOM	room_id (PK), hostel_id (FK), room_no, room_type (CHECK: AC/NON_AC), (hostel_id, room_no) UNIQUE	Stores room details for each hostel.
BED	bed_id (PK), room_id (FK), bed_no, service_status (DEFAULT: AVAILABLE, CHECK: AVAILABLE/OUT_OF_SERVICE), (room_id, bed_no) UNIQUE	Stores individual beds present in each room.
ALLOCATION	allocation_id (PK), student_id (FK), bed_id (FK), start_date, end_date	Stores allotment details of students to beds along with their stay duration.
FEE_CHARGE	charge_id (PK), allocation_id (FK), billing_period, charge_date, due_date, amount (CHECK > 0), (allocation_id, billing_period) UNIQUE	Stores fee charges generated for each allocation.
PAYMENT	payment_id (PK), charge_id (FK), payment_date, amount (CHECK > 0), payment_mode (CHECK: UPI/CARD/BANK/CASH), receipt_no (UNIQUE)	Stores payments made against fee charges.
ATTENDANCE	attendance_id (PK), allocation_id (FK), attendance_date, attendance_status (CHECK: PRESENT/ABSENT/APPROVED_LEAVE), (allocation_id, attendance_date) UNIQUE	Stores daily attendance records of residents.
COMPLAINT	complaint_id (PK), student_id (FK), room_id (FK), category (CHECK: PLUMBING/ELECTRICAL/FURNITURE/CLEANING/NETWORK), description, raised_date, status (DEFAULT: OPEN, CHECK: OPEN/IN_PROGRESS/RESOLVED), resolved_date	Stores maintenance and service complaints raised by students.
MAINTENANCE_EXPENSE	expense_id (PK), complaint_id (FK), expense_date, amount (CHECK >= 0), description	Stores expenses incurred for resolving complaints.
VISITOR_VISIT	visit_id (PK), allocation_id (FK), visitor_name, relationship, entry_time, exit_time	Stores visitor entry and exit details for residents.
DISCIPLINARY_INCIDENT	incident_id (PK), allocation_id (FK), incident_date, incident_type, severity (CHECK: LOW/MEDIUM/HIGH), action_taken	Stores records of disciplinary incidents.
REPORT_CONFIG	config_id (PK, CHECK = 1), window_start, as_of_date, occupancy_benchmark (CHECK BETWEEN 0 AND 100)	Stores configuration values used for reporting.
C6_WRITE_LOCK	lock_name (PK), token (DEFAULT 0)	Used internally by triggers to coordinate write operations across logical domains.

2.3.2 Relationships and Cardinality

Relationship	Cardinality	Explanation
HOSTEL - ROOM	1 : M	A hostel can have multiple rooms, but each room belongs to exactly one hostel.
ROOM - BED	1 : M	A room can contain multiple beds, but each bed belongs to exactly one room.
STUDENT - ALLOCATION	1 : M	A student can have one or more allocations over time (current and past), however not overlapping.
BED - ALLOCATION	1 : M	A bed can have a history of allocations over time, but only one active allocation at a given time.
ALLOCATION - FEE_CHARGE	1 : M	A single allocation can have multiple fee charges for different billing periods.

FEE_CHARGE - PAYMENT	1 : M	A single fee charge can have multiple payments made against it until it is fully paid.
ALLOCATION - ATTENDANCE	1 : M	Each allocation can have multiple attendance records, one per day.
STUDENT - COMPLAINT	1 : M	A student can raise multiple complaints.
ROOM - COMPLAINT	1 : M	Complaints are associated with the room they were raised for.
COMPLAINT - MAINTENANCE_EXPENSE	1 : M	A single complaint can incur one or more maintenance expenses.
ALLOCATION - VISITOR_VISIT	1 : M	A resident (via their allocation) can have multiple visitor visits recorded.
ALLOCATION - DISCIPLINARY_INCIDENT	1 : M	A resident can be involved in multiple disciplinary incidents through their allocation.

3. Database Design

3.1 ER Diagram

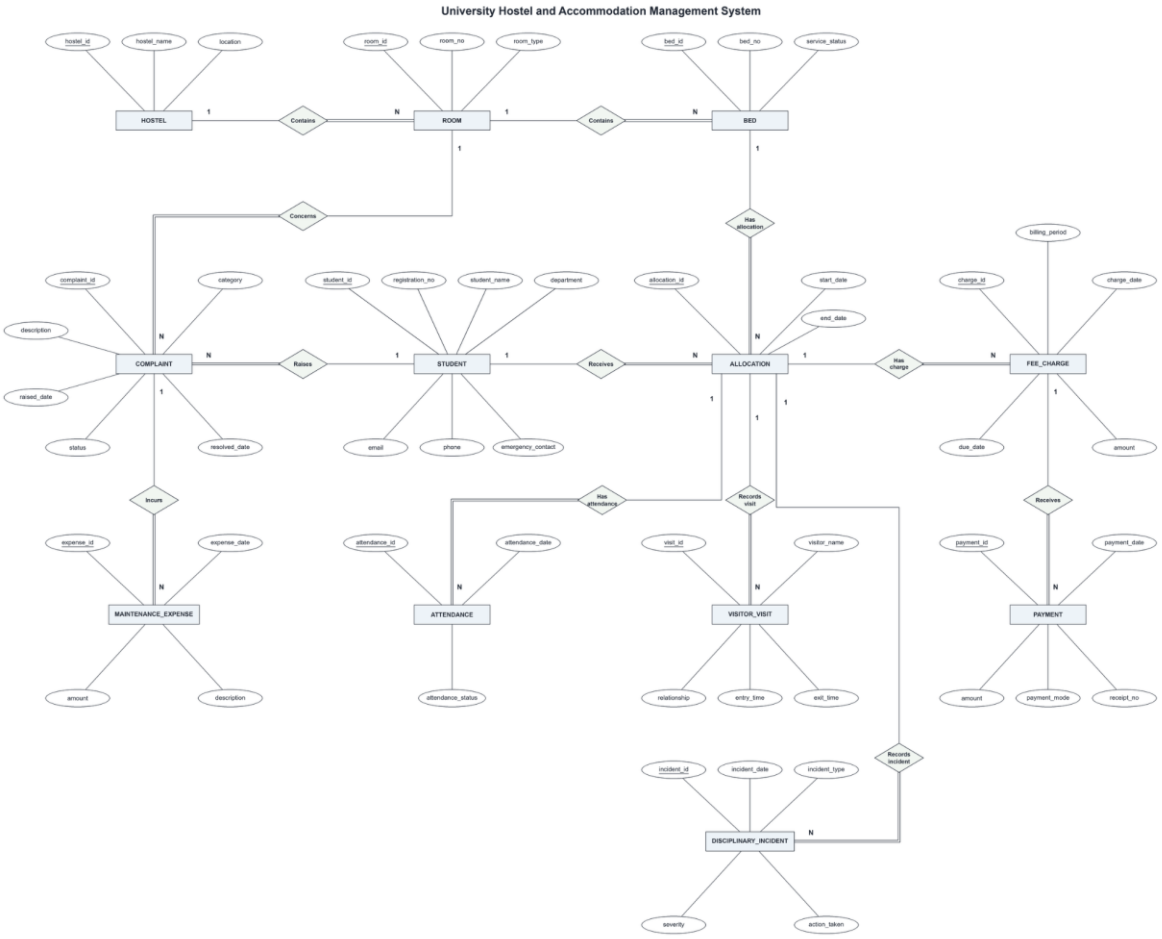


Figure 3.1: Entity–Relationship (ER) Diagram

3.2 Relational Schema

C6_WRITE_LOCK
C6_WRITE_LOCK (lock_name VARCHAR2(20), token NUMBER DEFAULT 0 NOT NULL)
Primary Key: lock_name

REPORT_CONFIG
REPORT_CONFIG (config_id NUMBER, window_start DATE NOT NULL, as_of_date DATE NOT NULL, occupancy_benchmark NUMBER(5,2) DEFAULT 90 NOT NULL)
Primary Key: config_id

STUDENT
STUDENT (student_id NUMBER, registration_no VARCHAR2(20) NOT NULL, student_name VARCHAR2(100) NOT NULL, department VARCHAR2(40) NOT NULL, email VARCHAR2(120) NOT NULL, phone VARCHAR2(20), emergency_contact VARCHAR2(20))
Primary Key: student_id

HOSTEL
HOSTEL (hostel_id NUMBER, hostel_name VARCHAR2(60) NOT NULL, location VARCHAR2(100) NOT NULL)
Primary Key: hostel_id

ROOM
ROOM (room_id NUMBER, hostel_id NUMBER NOT NULL, room_no VARCHAR2(10) NOT NULL, room_type VARCHAR2(20) NOT NULL)
Primary Key: room_id
Foreign Key: hostel_id REFERENCES HOSTEL(hostel_id)

BED
BED (bed_id NUMBER, room_id NUMBER NOT NULL, bed_no VARCHAR2(10) NOT NULL, service_status VARCHAR2(20) DEFAULT 'AVAILABLE' NOT NULL)
Primary Key: bed_id
Foreign Key: room_id REFERENCES ROOM(room_id)

ALLOCATION
ALLOCATION (allocation_id NUMBER, student_id NUMBER NOT NULL, bed_id NUMBER NOT NULL, start_date DATE NOT NULL, end_date DATE)
Primary Key: allocation_id
Foreign Key: student_id REFERENCES STUDENT(student_id)
Foreign Key: bed_id REFERENCES BED(bed_id)

FEE_CHARGE
FEE_CHARGE (charge_id NUMBER, allocation_id NUMBER NOT NULL, billing_period VARCHAR2(30) NOT NULL, charge_date DATE NOT NULL, due_date DATE NOT NULL, amount NUMBER(12,2) NOT NULL)
Primary Key: charge_id
Foreign Key: allocation_id REFERENCES ALLOCATION(allocation_id)

PAYMENT
PAYMENT (payment_id NUMBER, charge_id NUMBER NOT NULL, payment_date DATE NOT NULL, amount NUMBER(12,2) NOT NULL, payment_mode VARCHAR2(20) NOT NULL, receipt_no VARCHAR2(40) NOT NULL)
Primary Key: payment_id
Foreign Key: charge_id REFERENCES FEE_CHARGE(charge_id)

ATTENDANCE
ATTENDANCE (attendance_id NUMBER, allocation_id NUMBER NOT NULL, attendance_date DATE NOT NULL, attendance_status VARCHAR2(20) NOT NULL)
Primary Key: attendance_id
Foreign Key: allocation_id REFERENCES ALLOCATION(allocation_id)

COMPLAINT
COMPLAINT (complaint_id NUMBER, student_id NUMBER NOT NULL, room_id NUMBER NOT NULL, category VARCHAR2(30) NOT NULL, description VARCHAR2(500) NOT NULL, raised_date DATE NOT NULL, status VARCHAR2(20) DEFAULT 'OPEN' NOT NULL, resolved_date DATE)

Primary Key: complaint_id
Foreign Key: student_id REFERENCES STUDENT(student_id)
Foreign Key: room_id REFERENCES ROOM(room_id)

MAINTENANCE_EXPENSE
MAINTENANCE_EXPENSE (expense_id NUMBER, complaint_id NUMBER NOT NULL, expense_date DATE NOT NULL, amount NUMBER(12,2) NOT NULL, description VARCHAR2(200) NOT NULL)
Primary Key: expense_id
Foreign Key: complaint_id REFERENCES COMPLAINT(complaint_id)

VISITOR_VISIT
VISITOR_VISIT (visit_id NUMBER, allocation_id NUMBER NOT NULL, visitor_name VARCHAR2(100) NOT NULL, relationship VARCHAR2(30) NOT NULL, entry_time TIMESTAMP NOT NULL, exit_time TIMESTAMP)
Primary Key: visit_id
Foreign Key: allocation_id REFERENCES ALLOCATION(allocation_id)

DISCIPLINARY_INCIDENT
DISCIPLINARY_INCIDENT (incident_id NUMBER, allocation_id NUMBER NOT NULL, incident_date DATE NOT NULL, incident_type VARCHAR2(60) NOT NULL, severity VARCHAR2(10) NOT NULL, action_taken VARCHAR2(200) NOT NULL)
Primary Key: incident_id
Foreign Key: allocation_id REFERENCES ALLOCATION(allocation_id)

4. Database Implementation

4.1 Table Creation Using SQL

The University Hostel and Accommodation Management System was implemented using Oracle SQL. The database consists of 14 tables designed to manage student details, hostel information, room and bed availability, accommodation allocations, fee transactions, attendance, maintenance complaints, visitors, and disciplinary records.

The tables created are C6_WRITE_LOCK, REPORT_CONFIG, STUDENT, HOSTEL, ROOM, BED, ALLOCATION, FEE_CHARGE, PAYMENT, ATTENDANCE, COMPLAINT, MAINTENANCE_EXPENSE, VISITOR_VISIT, and DISCIPLINARY_INCIDENT.

Each table contains appropriate attributes and data types based on its purpose. The tables are interconnected through primary and foreign keys to maintain relationships between students, hostels, rooms, allocations, and other administrative records. Additional constraints, indexes, and triggers support data integrity and the enforcement of hostel management rules.

```sql
CREATE TABLE student (
    student_id NUMBER PRIMARY KEY,
    registration_no VARCHAR2(20) NOT NULL UNIQUE,
    student_name VARCHAR2(100) NOT NULL,
    department VARCHAR2(40) NOT NULL,
    email VARCHAR2(120) NOT NULL UNIQUE,
    phone VARCHAR2(20)
);

ALTER TABLE student ADD emergency_contact VARCHAR2(20);

CREATE TABLE hostel (
    hostel_id NUMBER PRIMARY KEY,
    hostel_name VARCHAR2(60) NOT NULL UNIQUE,
    location VARCHAR2(100) NOT NULL
);

CREATE TABLE room (
    room_id NUMBER PRIMARY KEY,
    hostel_id NUMBER NOT NULL REFERENCES hostel(hostel_id),
    room_no VARCHAR2(10) NOT NULL,
    room_type VARCHAR2(20) NOT NULL CHECK (room_type IN ('AC', 'NON_AC')),
    CONSTRAINT uq_room_hostel UNIQUE (hostel_id, room_no)
);
```

Figure 4.1: Core Table Creation Using SQL — STUDENT, HOSTEL, and ROOM

```sql
CREATE TABLE bed (
    bed_id NUMBER PRIMARY KEY,
    room_id NUMBER NOT NULL REFERENCES room(room_id),
    bed_no VARCHAR2(10) NOT NULL,
    service_status VARCHAR2(20) DEFAULT 'AVAILABLE' NOT NULL,
    CONSTRAINT ck_bed_service CHECK (service_status IN ('AVAILABLE', 'OUT_OF_SERVICE')),
    CONSTRAINT uq_bed_room UNIQUE (room_id, bed_no)
);

CREATE TABLE allocation (
    allocation_id NUMBER PRIMARY KEY,
    student_id NUMBER NOT NULL REFERENCES student(student_id),
    bed_id NUMBER NOT NULL REFERENCES bed(bed_id),
    start_date DATE NOT NULL,
    end_date DATE,
    CHECK (start_date = TRUNC(start_date)),
    CHECK (end_date = TRUNC(end_date)),
    CHECK (end_date IS NULL OR end_date > start_date)
);
```

Figure 4.2: Accommodation Table Creation Using SQL — BED and ALLOCATION

```sql
CREATE TABLE fee_charge (
    charge_id NUMBER PRIMARY KEY,
    allocation_id NUMBER NOT NULL REFERENCES allocation(allocation_id),
    billing_period VARCHAR2(30) NOT NULL,
    charge_date DATE NOT NULL,
    due_date DATE NOT NULL,
    amount NUMBER(12,2) NOT NULL CHECK (amount > 0),
    CHECK (due_date >= charge_date),
    CONSTRAINT uq_fee_period UNIQUE (allocation_id, billing_period)
);

CREATE TABLE payment (
    payment_id NUMBER PRIMARY KEY,
    charge_id NUMBER NOT NULL REFERENCES fee_charge(charge_id),
    payment_date DATE NOT NULL,
    amount NUMBER(12,2) NOT NULL CHECK (amount > 0),
    payment_mode VARCHAR2(20) NOT NULL,
    receipt_no VARCHAR2(40) NOT NULL UNIQUE,
    CHECK (payment_mode IN ('UPI', 'CARD', 'BANK', 'CASH'))
);
```

Figure 4.3: Financial Table Creation Using SQL — FEE_CHARGE and PAYMENT

```sql
CREATE TABLE attendance (
    attendance_id NUMBER PRIMARY KEY,
    allocation_id NUMBER NOT NULL REFERENCES allocation(allocation_id),
    attendance_date DATE NOT NULL,
    attendance_status VARCHAR2(20) NOT NULL,
    CHECK (attendance_date = TRUNC(attendance_date)),
    CHECK (attendance_status IN ('PRESENT', 'ABSENT', 'APPROVED_LEAVE')),
    CONSTRAINT uq_attendance_day UNIQUE (allocation_id, attendance_date)
);

CREATE TABLE complaint (
    complaint_id NUMBER PRIMARY KEY,
    student_id NUMBER NOT NULL REFERENCES student(student_id),
    room_id NUMBER NOT NULL REFERENCES room(room_id),
    category VARCHAR2(30) NOT NULL,
    description VARCHAR2(500) NOT NULL,
    raised_date DATE NOT NULL,
    status VARCHAR2(20) DEFAULT 'OPEN' NOT NULL,
    resolved_date DATE,
    CHECK (category IN ('PLUMBING', 'ELECTRICAL', 'FURNITURE', 'CLEANING', 'NETWORK')),
    CHECK (status IN ('OPEN', 'IN_PROGRESS', 'RESOLVED')),
    CHECK (resolved_date IS NULL OR resolved_date >= raised_date),
    CHECK ((status = 'RESOLVED' AND resolved_date IS NOT NULL)
        OR (status IN ('OPEN', 'IN_PROGRESS') AND resolved_date IS NULL))
);
```

Figure 4.4: Administrative Table Creation Using SQL — ATTENDANCE and COMPLAINT

4.2 Integrity Constraints

Integrity constraints were implemented in the University Hostel and Accommodation Management System using Oracle SQL to maintain data accuracy, consistency, and referential integrity. These constraints ensure that student records, room allocations, fee transactions, attendance, and other administrative data follow the defined business rules.

The following types of integrity constraints are used:
• Primary Key Constraints: Uniquely identify each record in a table and prevent duplicate identifiers.
• Foreign Key Constraints: Establish relationships between related tables and prevent references to records that do not exist.
• NOT NULL Constraints: Ensure that essential attributes cannot be left empty.
• UNIQUE Constraints: Prevent duplicate values in attributes or combinations of attributes that must remain unique.
• CHECK Constraints: Restrict values to valid conditions, such as acceptable amounts, statuses, or dates.
• Triggers: Enforce additional business rules that require validation beyond ordinary table constraints, including rules related to accommodation allocations, attendance, payments, fee charges, and bed availability.
Together, these mechanisms help prevent invalid records, maintain consistent relationships between tables, and support reliable hostel administration.

4.3 Data Insertion

Sample data was inserted into the database using Oracle SQL to demonstrate the functioning of the University Hostel and Accommodation Management System. The data includes student details, hostel information, rooms, beds, accommodation allocations, fee charges, payments, attendance records, maintenance complaints, maintenance expenses, visitor visits, and disciplinary incidents.
The sample data was designed to represent realistic hostel management scenarios and establish relationships between the database tables. Records were inserted in an appropriate order to satisfy foreign key constraints and maintain referential integrity.
PL/SQL loops were used to generate room and bed records efficiently, while SQL INSERT statements populated the remaining tables. The database also includes historical accommodation records and a bed marked as out of service to support testing of allocation history and bed availability rules.
This populated database provides sufficient interrelated data for testing database constraints, verifying business rules, and generating analytical reports on hostel occupancy, fee collection, attendance, maintenance complaints, and student support risk.

4.4 Data Modification and Deletion

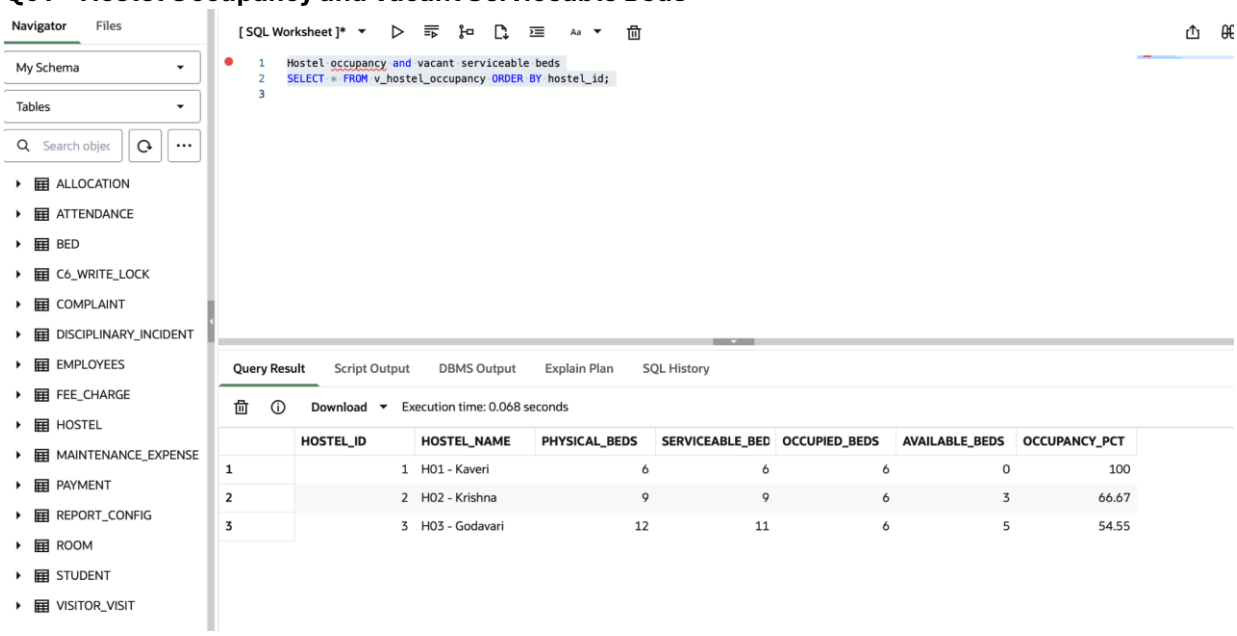
The database supports modification and deletion of records using the SQL UPDATE and DELETE commands. These operations demonstrate how existing hostel management information can be maintained when changes occur.
The UPDATE command is used to modify existing records, such as changing visitor details or updating the status of a record. The DELETE command is used to remove records when deletion is permitted by the database constraints.
The implementation also includes a demonstration of data manipulation operations in the verification script, 05_verify.sql. This script performs insertion, modification, and deletion operations on visitor records to demonstrate basic data management functionality.
Foreign key constraints and trigger-based business rules help prevent operations that could compromise data consistency. For example, the database includes validation rules relating to accommodation allocations, attendance records, fee payments, and bed availability. These safeguards ensure that modifications and deletions do not violate the defined relationships and business rules.
Thus, the use of SQL data manipulation commands, together with integrity constraints and triggers, supports controlled and consistent management of hostel records.

5. SQL Queries and Outputs

The Hostel Management System uses SQL queries to retrieve, process, and analyse information stored across multiple related tables. The queries demonstrate filtering, joins, aggregate functions, grouping, sorting, subqueries, correlated subqueries, and views. They generate reports that support hostel occupancy monitoring, attendance analysis, fee management, maintenance tracking, disciplinary monitoring, and administrative decision-making.

5.1 Hostel Occupancy and Room Utilization

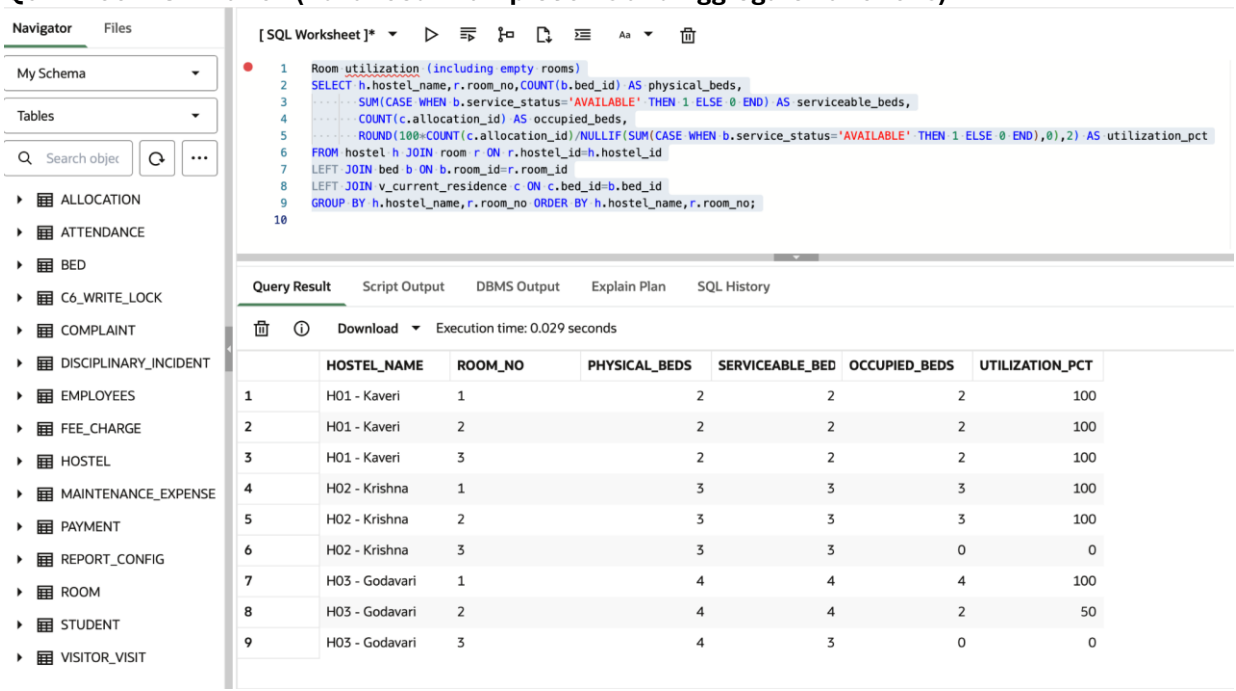
Q01 – Hostel Occupancy and Vacant Serviceable Beds



	HOSTEL_ID	HOSTEL_NAME	PHYSICAL_BEDS	SERVICEABLE_BED	OCCUPIED_BEDS	AVAILABLE_BEDS	OCCUPANCY_PCT
1	1	H01 - Kaveri	6	6	6	0	100
2	2	H02 - Krishna	9	9	6	3	66.67
3	3	H03 - Godavari	12	11	6	5	54.55

This query retrieves hostel-wise occupancy information, including physical beds, serviceable beds, occupied beds, available beds, and occupancy percentage. It uses the V_HOSTEL_OCCUPANCY view to present summarized accommodation statistics.

Q02 – Room Utilization (Advanced: Multiple Joins and Aggregate Functions)

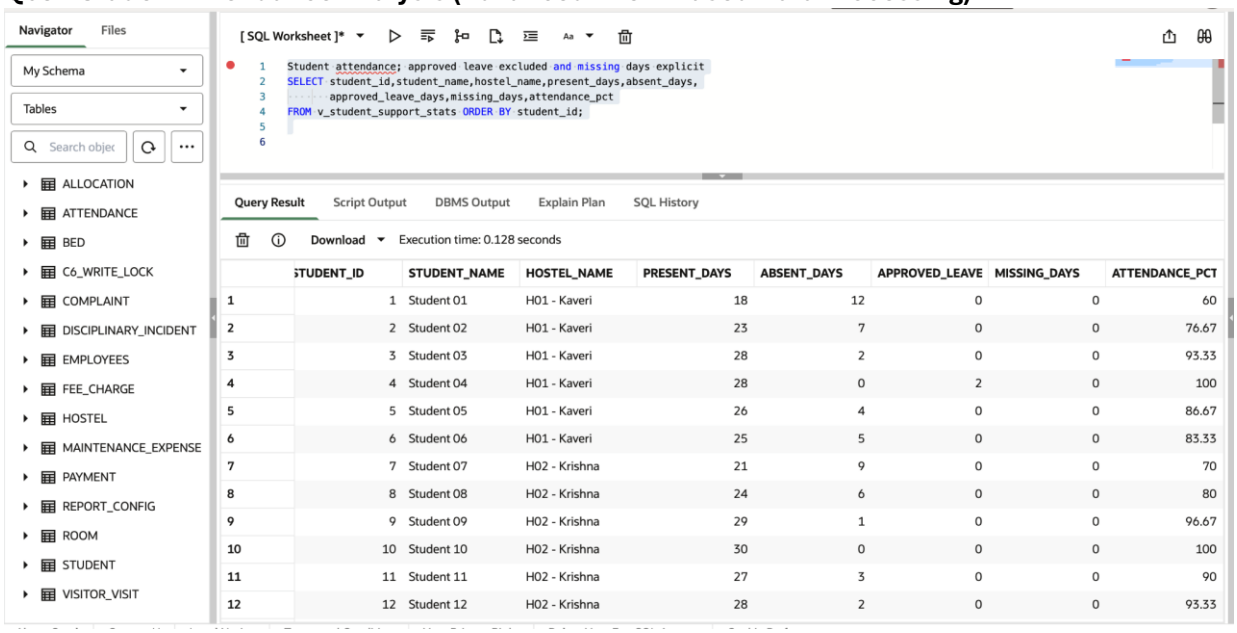


	HOSTEL_NAME	ROOM_NO	PHYSICAL_BEDS	SERVICEABLE_BED	OCCUPIED_BEDS	UTILIZATION_PCT
1	H01 - Kaveri	1	2	2	2	100
2	H01 - Kaveri	2	2	2	2	100
3	H01 - Kaveri	3	2	2	2	100
4	H02 - Krishna	1	3	3	3	100
5	H02 - Krishna	2	3	3	3	100
6	H02 - Krishna	3	3	3	0	0
7	H03 - Godavari	1	4	4	4	100
8	H03 - Godavari	2	4	4	2	50
9	H03 - Godavari	3	4	3	0	0

This query calculates the utilization of individual rooms by retrieving physical beds, serviceable beds, occupied beds, and utilization percentages. It uses joins and aggregate functions, including COUNT, SUM, and ROUND. Left joins allow empty rooms to be included in the results.

5.2 Student Attendance Analysis

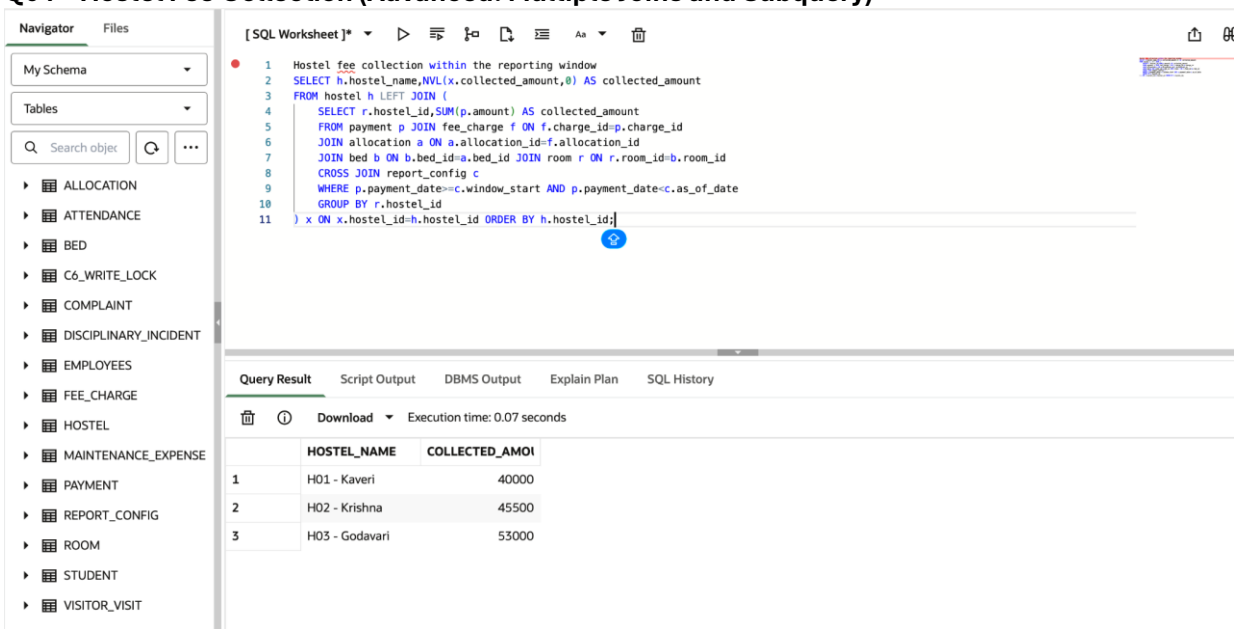
Q03 – Student Attendance Analysis (Advanced: View-Based Data Processing)



This query retrieves student-wise attendance statistics, including present days, absent days, approved leave days, missing days, and attendance percentage. It uses the V_STUDENT_SUPPORT_STATS view, which combines attendance information with student and accommodation data. Approved leave is excluded from the attendance denominator, while missing attendance days are reported separately.

5.3 Hostel Fee Collection and Outstanding Balances

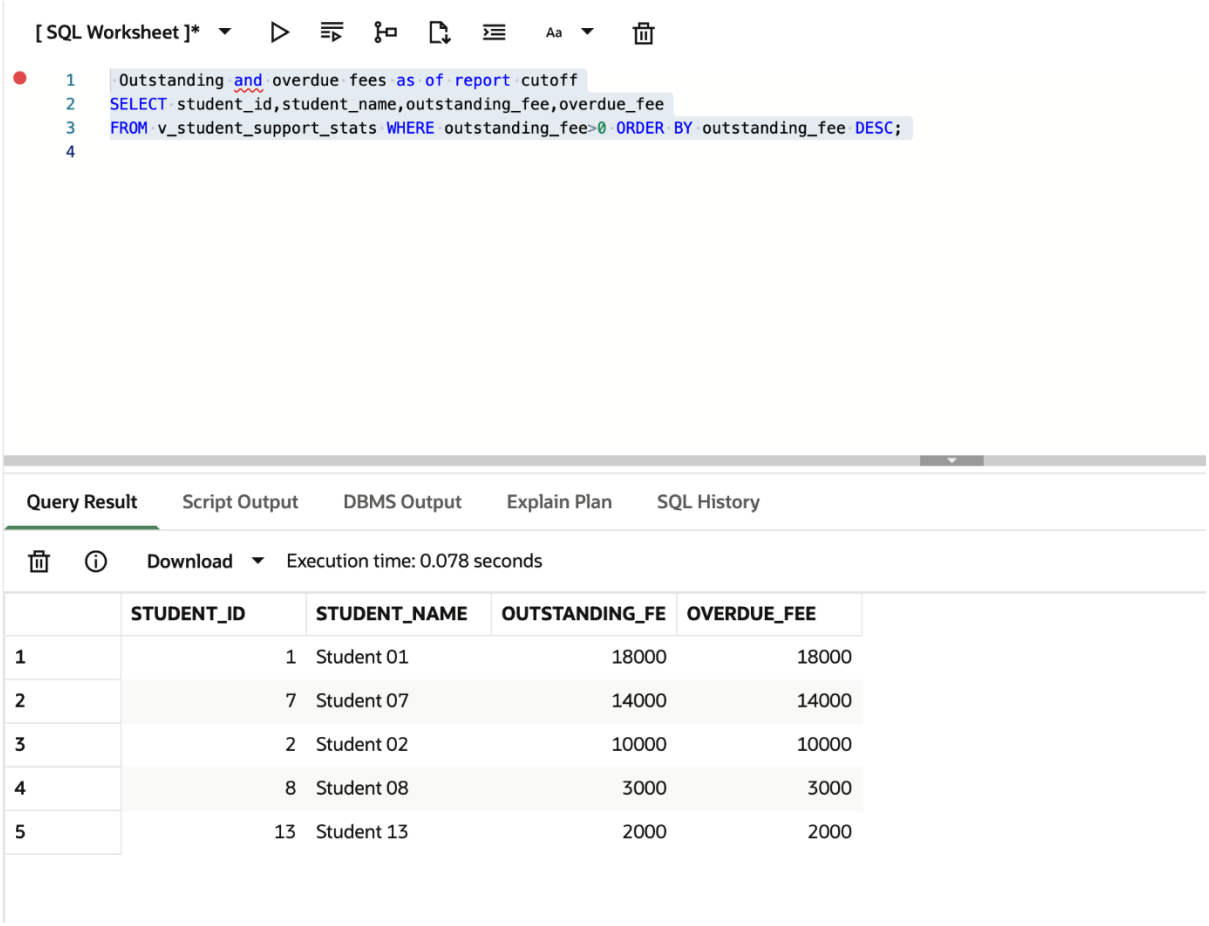
Q04 – Hostel Fee Collection (Advanced: Multiple Joins and Subquery)



This query calculates the amount collected by each hostel during the configured reporting window. An inner query joins payment, fee charge, allocation, bed, and room information, groups payment

amounts by hostel, and calculates the total collection. The outer query uses a left join to retain hostels with no collections during the reporting period.

Q05 – Outstanding and Overdue Fees (Advanced: View-Based Analysis)

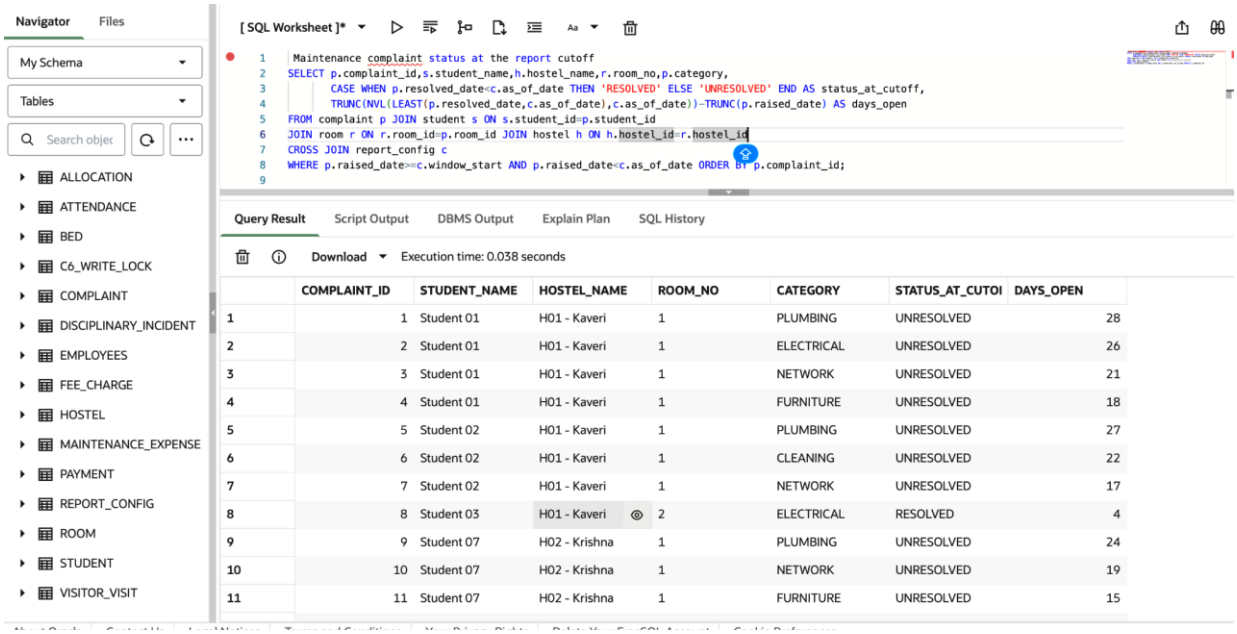


	STUDENT_ID	STUDENT_NAME	OUTSTANDING_FE	OVERDUE_FEE
1	1	Student 01	18000	18000
2	7	Student 07	14000	14000
3	2	Student 02	10000	10000
4	8	Student 08	3000	3000
5	13	Student 13	2000	2000

This query identifies students with outstanding hostel fees and displays their outstanding and overdue amounts. It uses the V_STUDENT_SUPPORT_STATS view and sorts students by outstanding balance in descending order to highlight larger dues.

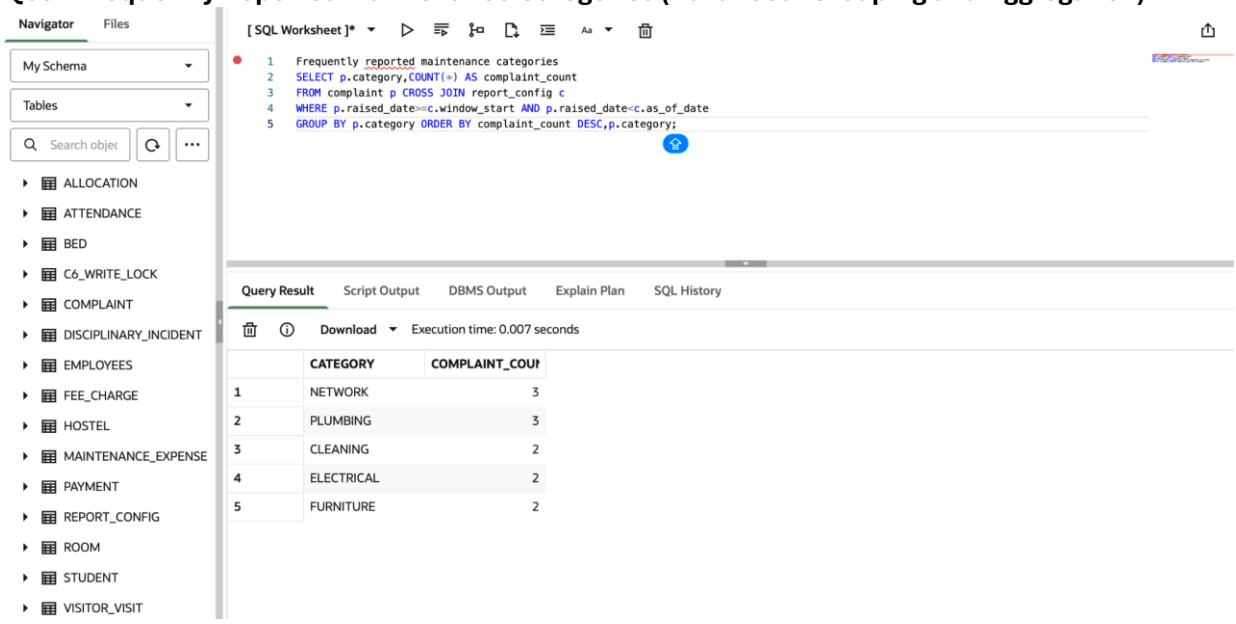
5.4 Maintenance Complaints and Expenses

Q06 – Maintenance Complaint Status at the Reporting Cutoff (Advanced: Multiple Joins and Conditional Expressions)



This query retrieves maintenance complaint details, including the student, hostel, room, complaint category, status at the reporting cutoff, and the number of days the complaint has remained open. It uses joins and a CASE expression to determine the complaint status relative to the configured reporting date.

Q08 – Frequently Reported Maintenance Categories (Advanced: Grouping and Aggregation)



This query identifies frequently reported maintenance problem categories by counting complaints within the reporting window. The results are grouped by category and sorted by complaint count, helping the administration identify recurring maintenance issues.

Q15 – Above-Average Maintenance Expenditure (Advanced: Aggregate Subquery)

[SQL Worksheet]*

```
1   Above-average maintenance expenditure (includes zero-cost hostels)
2   SELECT hostel_name,maintenance_cost FROM v_hostel_performance
3   WHERE maintenance_cost>(SELECT AVG(maintenance_cost) FROM v_hostel_performance);
```

Query Result | Script Output | DBMS Output | Explain Plan | SQL History

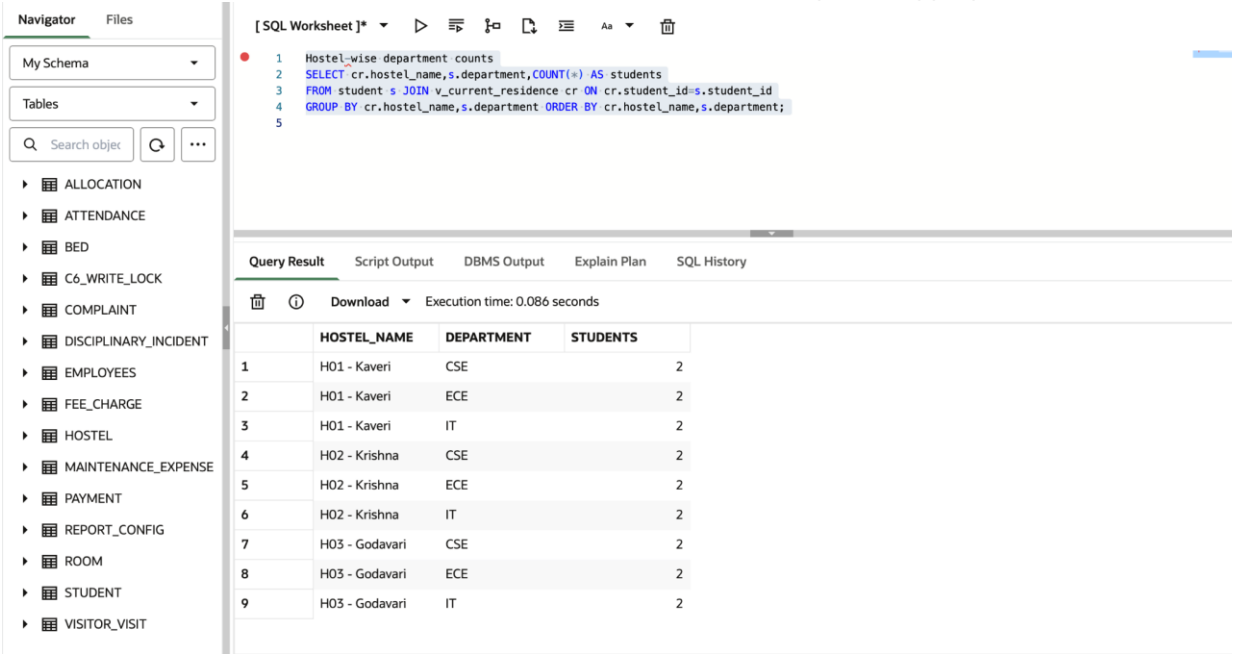
Download | Execution time: 0.458 seconds

	HOSTEL_NAME	MAINTENANCE_COST
1	H01 - Kaveri	3800

This query identifies hostels whose maintenance expenditure exceeds the average expenditure across all hostels. A subquery calculates the average maintenance cost, and the outer query compares each hostel's expenditure with that benchmark. The analysis includes hostels with zero maintenance expenditure through the V_HOSTEL_PERFORMANCE view.

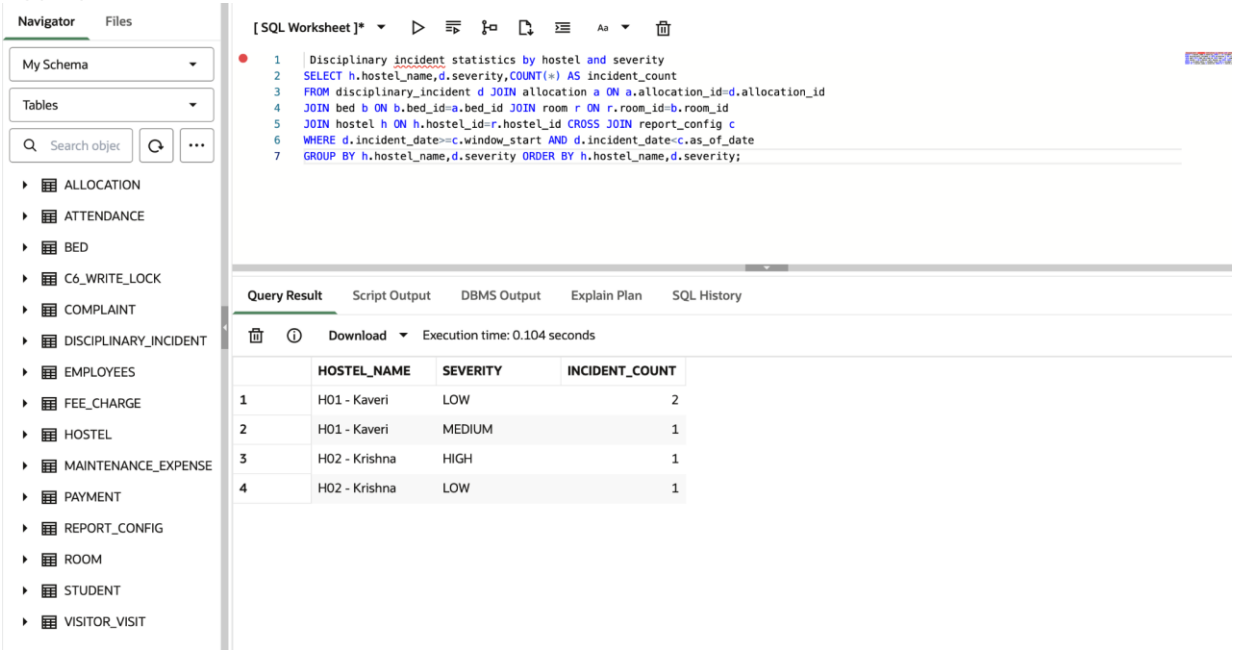
5.5 Hostel-Wise and Disciplinary Statistics

Q07 – Hostel-Wise Department Counts (Advanced: Join, Grouping, and Aggregation)



This query calculates the number of students from each department residing in each hostel. It joins student information with the V_CURRENT_RESIDENCE view and groups the results by hostel and department.

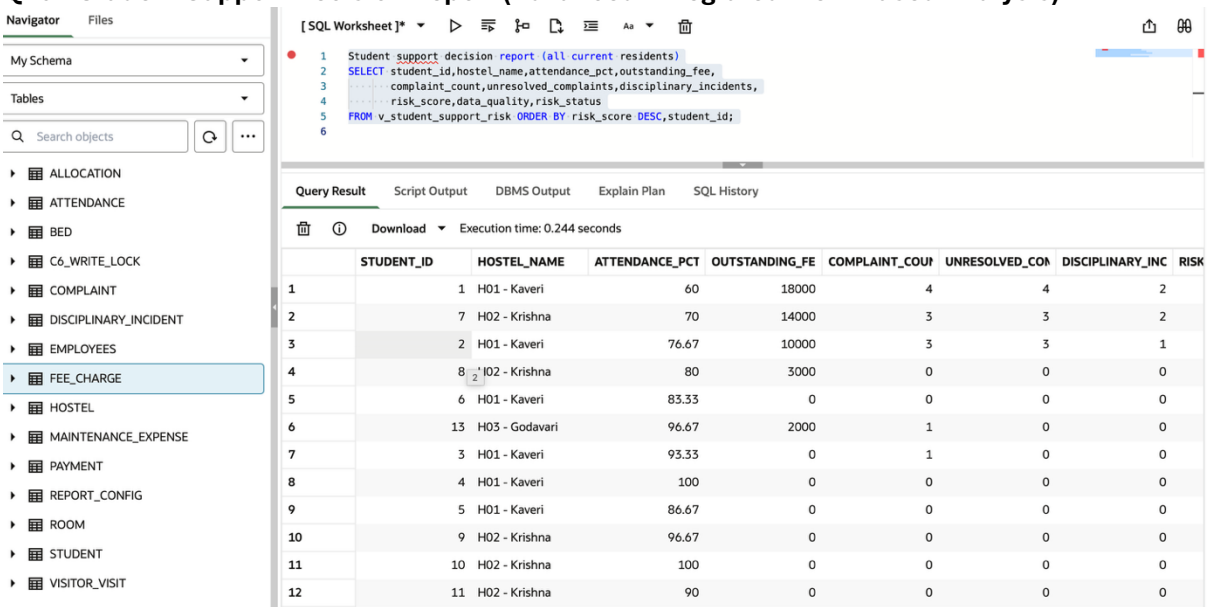
Q09 – Disciplinary Incident Statistics by Hostel and Severity (Advanced: Multiple Joins and Aggregation)

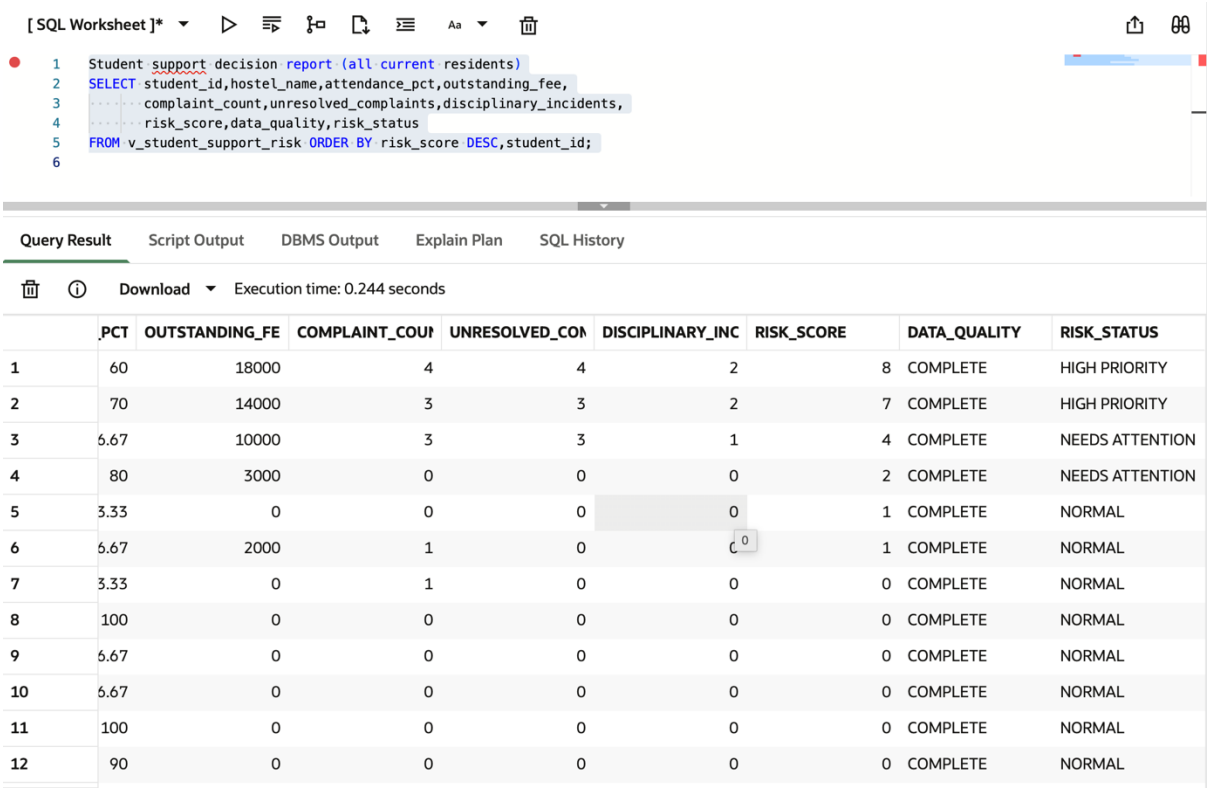


This query summarizes disciplinary incidents by hostel and severity level. It joins disciplinary incidents with allocations, beds, rooms, and hostels, then groups and counts the incidents. The results help administrators examine disciplinary patterns across accommodation facilities.

5.6 Student Support Risk and Administrative Attention

Q10 – Student Support Decision Report (Advanced: Integrated View-Based Analysis)





	PCT	OUTSTANDING_FE	COMPLAINT_COUI	UNRESOLVED_CON	DISCIPLINARY_INC	RISK_SCORE	DATA_QUALITY	RISK_STATUS
1	60	18000	4	4	2	8	COMPLETE	HIGH PRIORITY
2	70	14000	3	3	2	7	COMPLETE	HIGH PRIORITY
3	6.67	10000	3	3	1	4	COMPLETE	NEEDS ATTENTION
4	80	3000	0	0	0	2	COMPLETE	NEEDS ATTENTION
5	3.33	0	0	0	0	1	COMPLETE	NORMAL
6	6.67	2000	1	0	0	1	COMPLETE	NORMAL
7	3.33	0	1	0	0	0	COMPLETE	NORMAL
8	100	0	0	0	0	0	COMPLETE	NORMAL
9	6.67	0	0	0	0	0	COMPLETE	NORMAL
10	6.67	0	0	0	0	0	COMPLETE	NORMAL
11	100	0	0	0	0	0	COMPLETE	NORMAL
12	90	0	0	0	0	0	COMPLETE	NORMAL

This query generates a student support report for current residents. It displays attendance percentage, outstanding fees, complaint counts, unresolved complaints, disciplinary incidents, risk score, data-quality information, and risk status. The report uses the V_STUDENT_SUPPORT_RISK view to combine multiple indicators into one decision-support output.

Q11 – Above-Average Outstanding Fees (Advanced: Scalar Subquery)

[SQL Worksheet]*

```
1  Above-average outstanding fees (benchmark includes all current residents, including zero balances)
2  SELECT student_id,student_name,outstanding_fee
3  FROM v_student_support_risk
4  WHERE outstanding_fee>(SELECT AVG(outstanding_fee) FROM v_student_support_risk)
5  ORDER BY outstanding_fee DESC;
```

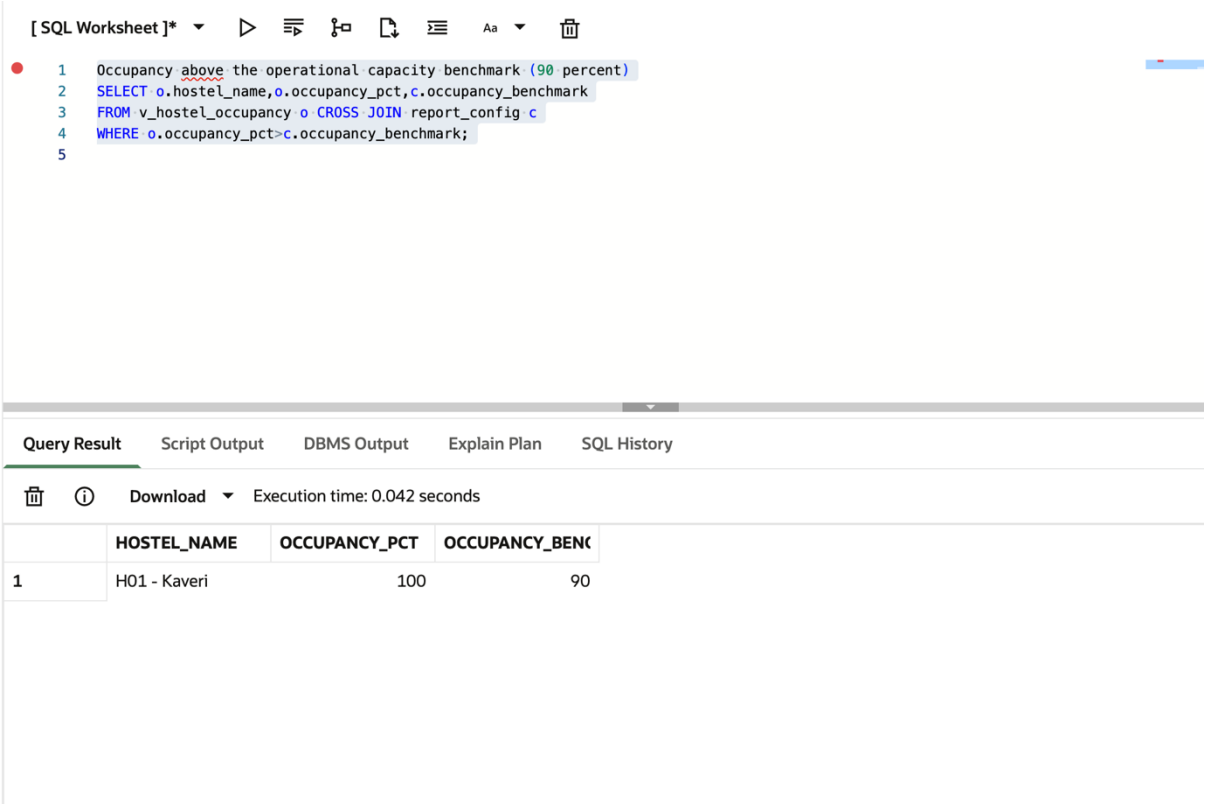
Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download Execution time: 0.248 seconds

	STUDENT_ID	STUDENT_NAME	OUTSTANDING_FE
1	1	Student 01	18000
2	7	Student 07	14000
3	2	Student 02	10000
4	8	Student 08	3000

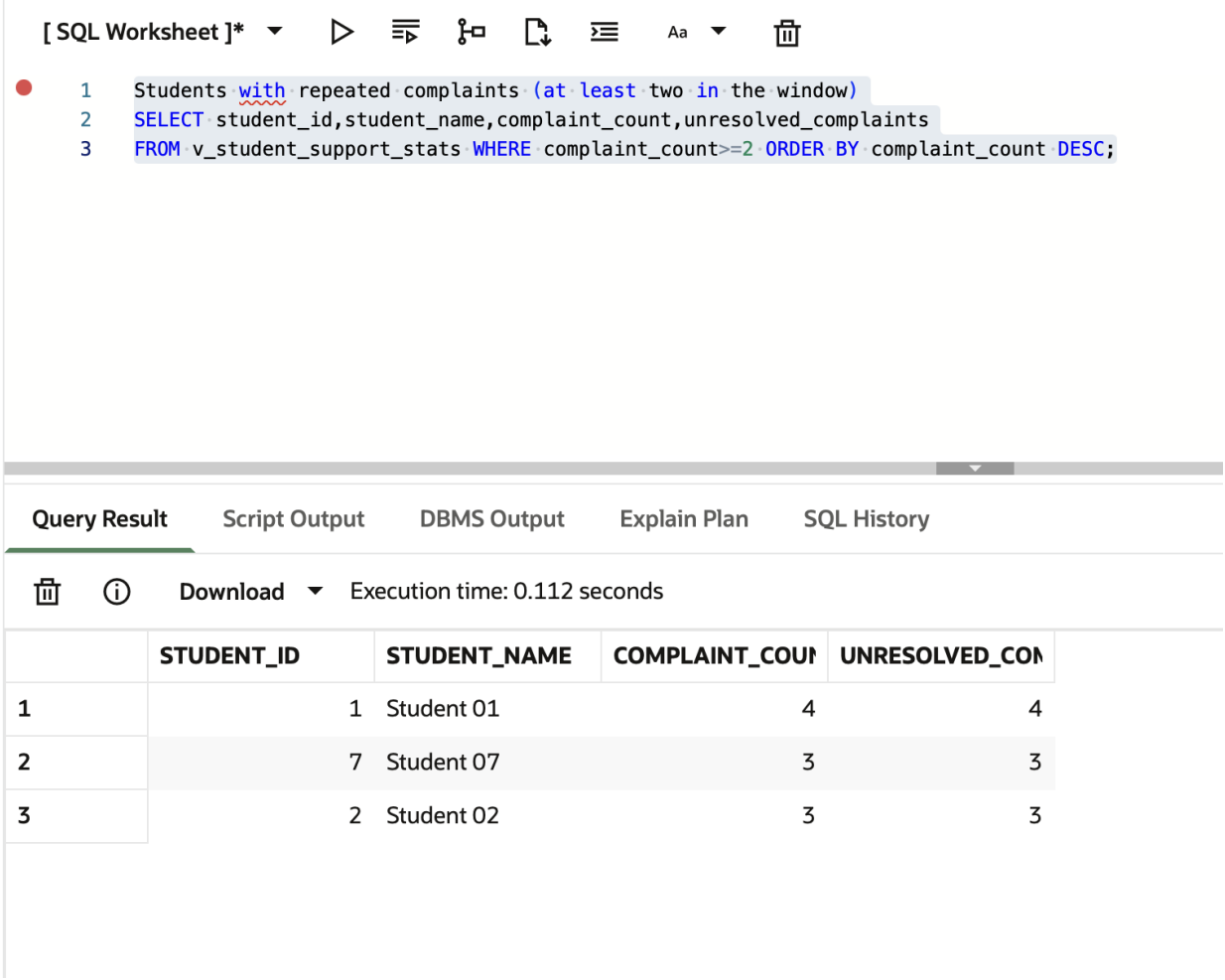
This query identifies students whose outstanding fees exceed the average outstanding balance among current residents. The subquery calculates the average outstanding amount, including residents with zero balances, and the outer query retrieves students whose balances exceed that average.

Q12 – Occupancy Above the Operational Capacity Benchmark (Advanced: Benchmark Comparison)



This query identifies hostels whose occupancy percentage exceeds the configured operational benchmark of 90%. It compares the occupancy percentage from V_HOSTEL_OCCUPANCY with the benchmark stored in REPORT_CONFIG.

Q13 – Students with Repeated Maintenance Complaints (Advanced: View-Based Filtering and Aggregation)



	STUDENT_ID	STUDENT_NAME	COMPLAINT_COUI	UNRESOLVED_CON
1	1	Student 01	4	4
2	7	Student 07	3	3
3	2	Student 02	3	3

This query identifies students with at least two maintenance complaints during the configured reporting window. It retrieves the complaint count and unresolved complaint count from V_STUDENT_SUPPORT_STATS, helping the administration identify students experiencing recurring maintenance problems.

Q14 – Attendance Below the Student's Own Hostel Average (Advanced: Correlated Subquery)

```
CORRELATED SUBQUERY: attendance below the student's own hostel average
SELECT s.student_id,s.student_name,s.hostel_name,s.attendance_pct
FROM v_student_support_stats s
WHERE s.hostel_id IS NOT NULL AND s.missing_days=0
AND s.attendance_pct < (
    SELECT AVG(peer.attendance_pct) FROM v_student_support_stats peer
    WHERE peer.hostel_id=s.hostel_id AND peer.missing_days=0
)
ORDER BY s.hostel_id,s.attendance_pct;
```

Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.392 seconds

	STUDENT_ID	STUDENT_NAME	HOSTEL_NAME	ATTENDANCE_PCT
1	1	Student 01	H01 - Kaveri	60
2	2	Student 02	H01 - Kaveri	76.67
3	6	Student 06	H01 - Kaveri	83.33
4	7	Student 07	H02 - Krishna	70
5	8	Student 08	H02 - Krishna	80
6	15	Student 15	H03 - Godavari	90

This query identifies eligible students whose attendance percentage is lower than the average attendance percentage of eligible students in the same hostel. The inner query references the hostel identifier from the outer query, making it a correlated subquery. Students with missing attendance days are excluded from this comparison.

Q16 – Repeated Disciplinary Incidents (Advanced: View-Based Filtering)

```
Repeated disciplinary incidents (at least two in the window)
SELECT student_id,student_name,disciplinary_incidents
FROM v_student_support_stats WHERE disciplinary_incidents>=2;
```

Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.056 seconds

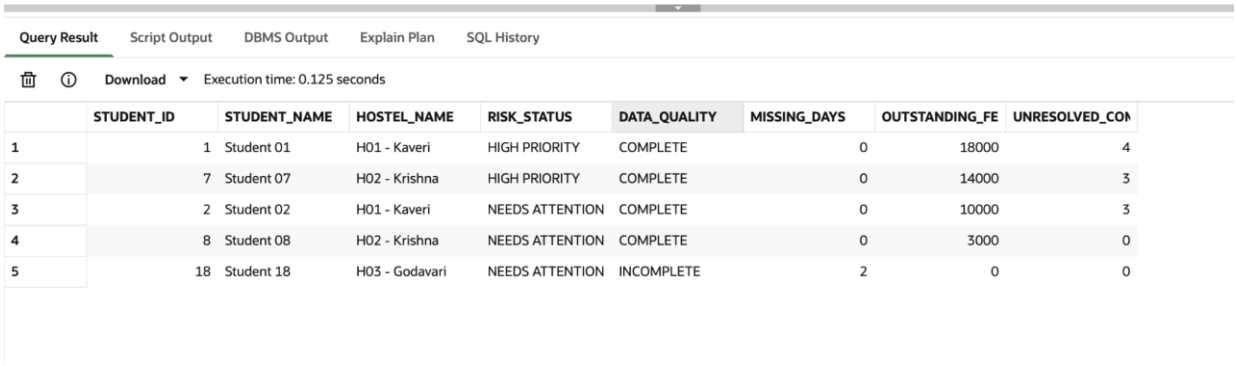
	STUDENT_ID	STUDENT_NAME	DISCIPLINARY_INCIDENTS
1	1	Student 01	2
2	7	Student 07	2

This query identifies students with at least two disciplinary incidents during the configured reporting window. It uses the disciplinary incident count in V_STUDENT_SUPPORT_STATS to identify cases that may require administrative review.

Q21 – Administrative Attention Queue (Advanced: Conditional Sorting and View-Based Filtering)
This query generates a prioritized list of current residents whose risk status is not NORMAL. It displays risk status, data-quality information, missing attendance days, outstanding fees, and unresolved complaints. A CASE expression places high-priority students first, followed by other students requiring attention, with additional sorting by risk score.



```
Administrative attention queue
SELECT student_id,student_name,hostel_name,risk_status,data_quality,
       missing_days,outstanding_fee,unresolved_complaints
FROM v_student_support_risk WHERE risk_status<>'NORMAL'
ORDER BY CASE risk_status WHEN 'HIGH PRIORITY' THEN 1 ELSE 2 END,risk_score DESC,student_id;
```



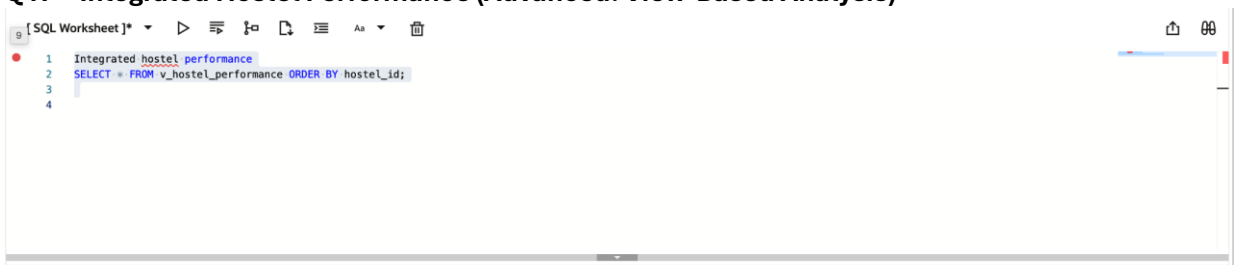
Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download — Execution time: 0.125 seconds

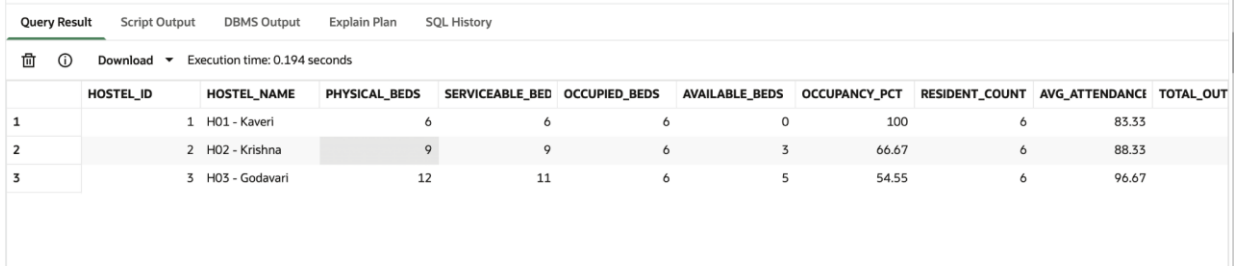
	STUDENT_ID	STUDENT_NAME	HOSTEL_NAME	RISK_STATUS	DATA_QUALITY	MISSING_DAYS	OUTSTANDING_FE	UNRESOLVED_COM
1	1	Student 01	H01 - Kaveri	HIGH PRIORITY	COMPLETE	0	18000	4
2	7	Student 07	H02 - Krishna	HIGH PRIORITY	COMPLETE	0	14000	3
3	2	Student 02	H01 - Kaveri	NEEDS ATTENTION	COMPLETE	0	10000	3
4	8	Student 08	H02 - Krishna	NEEDS ATTENTION	COMPLETE	0	3000	0
5	18	Student 18	H03 - Godavari	NEEDS ATTENTION	INCOMPLETE	2	0	0

5.7 Hostel Performance, Vacancies, and History

Q17 – Integrated Hostel Performance (Advanced: View-Based Analysis)



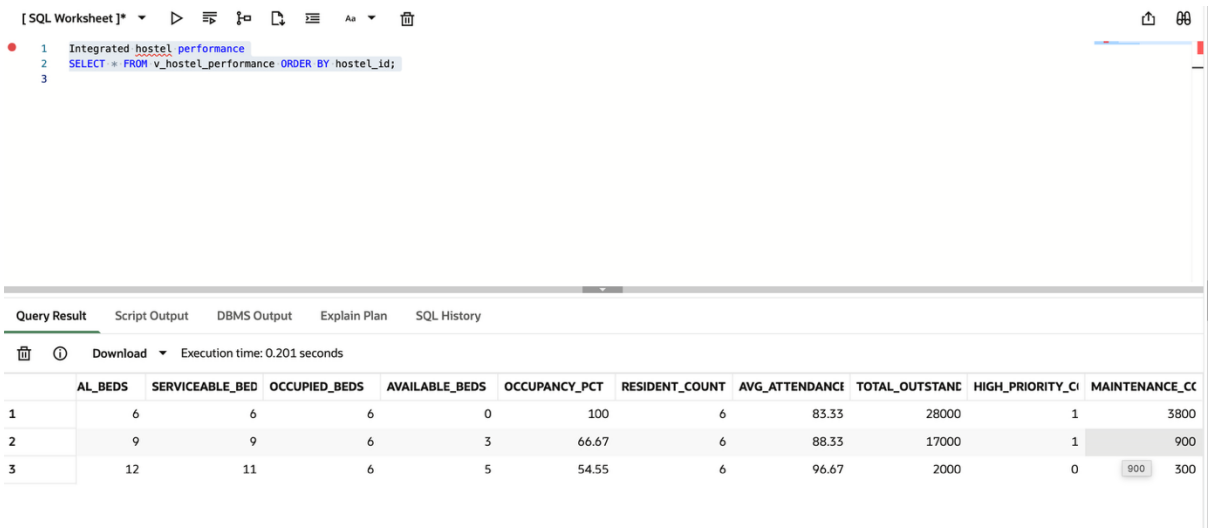
```
Integrated hostel performance
SELECT * FROM v_hostel_performance ORDER BY hostel_id;
```



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download — Execution time: 0.194 seconds

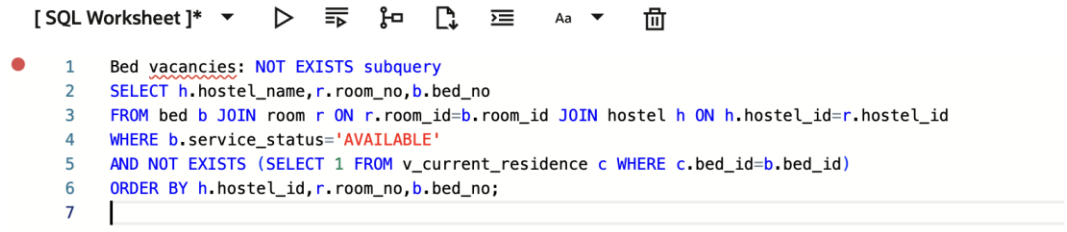
	HOSTEL_ID	HOSTEL_NAME	PHYSICAL_BEDS	SERVICEABLE_BED	OCCUPIED_BEDS	AVAILABLE_BEDS	OCCUPANCY_PCT	RESIDENT_COUNT	AVG_ATTENDANCE	TOTAL_OUT
1	1	H01 - Kaveri	6	6	6	0	100	6	83.33	
2	2	H02 - Krishna	9	9	6	3	66.67	6	88.33	
3	3	H03 - Godavari	12	11	6	5	54.55	6	96.67	



	AL_BEDS	SERVICEABLE_BED	OCCUPIED_BEDS	AVAILABLE_BEDS	OCCUPANCY_PCT	RESIDENT_COUNT	AVG_ATTENDANCE	TOTAL_OUTSTANC	HIGH_PRIORITY_C	MAINTENANCE_CC
1	6	6	6	0	100	6	83.33	28000	1	3800
2	9	9	6	3	66.67	6	88.33	17000	1	900
3	12	11	6	5	54.55	6	96.67	2000	0	900 300

This query retrieves the integrated performance indicators for each hostel using the V_HOSTEL_PERFORMANCE view. It provides a consolidated basis for comparing hostel occupancy and maintenance expenditure.

Q18 – Available Bed Vacancies (Advanced: NOT EXISTS Subquery)




```
1  Bed vacancies: NOT EXISTS subquery
2  SELECT h.hostel_name,r.room_no,b.bed_no
3  FROM bed b JOIN room r ON r.room_id=b.room_id JOIN hostel h ON h.hostel_id=r.hostel_id
4  WHERE b.service_status='AVAILABLE'
5  AND NOT EXISTS (SELECT 1 FROM v_current_residence c WHERE c.bed_id=b.bed_id)
6  ORDER BY h.hostel_id,r.room_no,b.bed_no;
7
```



	HOSTEL_NAME	ROOM_NO	BED_NO
1	H02 - Krishna	3	1
2	H02 - Krishna	3	2
3	H02 - Krishna	3	3
4	H03 - Godavari	2	3
5	H03 - Godavari	2	4
6	H03 - Godavari	3	1
7	H03 - Godavari	3	2
8	H03 - Godavari	3	3

This query identifies serviceable beds that do not have a current residence allocation. The NOT EXISTS subquery checks whether a bed appears in V_CURRENT_RESIDENCE. Beds without a matching allocation are returned as available vacancies.

Q19 – Visitor History (Advanced: Multiple Joins and Date Filtering)

[SQL Worksheet]*

```
1  Visitor history (including actual entry and exit times)
2  SELECT s.student_name,h.hostel_name,v.visitor_name,v.relationship,v.entry_time,v.exit_time
3  FROM visitor_visit v JOIN allocation a ON a.allocation_id=v.allocation_id
4  JOIN student s ON s.student_id=a.student_id
5  JOIN bed b ON b.bed_id=a.bed_id JOIN room r ON r.room_id=b.room_id
6  JOIN hostel h ON h.hostel_id=r.hostel_id CROSS JOIN report_config c
7  WHERE v.entry_time>=CAST(c.window_start AS TIMESTAMP) AND v.entry_time<CAST(c.as_of_date AS TIMESTAMP)
8  ORDER BY v.entry_time,s.student_id;
```

Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.026 seconds

	STUDENT_NAME	HOSTEL_NAME	VISITOR_NAME	RELATIONSHIP	ENTRY_TIME	EXIT_TIME
1	Student 01	H01 - Kaveri	Visitor 1	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
2	Student 02	H01 - Kaveri	Visitor 2	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
3	Student 03	H01 - Kaveri	Visitor 3	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
4	Student 04	H01 - Kaveri	Visitor 4	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
5	Student 05	H01 - Kaveri	Visitor 5	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
6	Student 06	H01 - Kaveri	Visitor 6	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
7	Student 07	H02 - Krishna	Visitor 7	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
8	Student 08	H02 - Krishna	Visitor 8	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
9	Student 09	H02 - Krishna	Visitor 9	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
10	Student 10	H02 - Krishna	Visitor 10	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(
11	Student 11	H02 - Krishna	Visitor 11	PARENT	2026-09-20T10:00:(	2026-09-20T11:00:(

This query retrieves visitor records with the associated student's name, hostel, visitor name, relationship, entry time, and exit time. It joins visitor records with allocations, students, beds, rooms, and hostels, and filters visits using the configured reporting window.

Q20 – Accommodation Allocation and Transfer History (Advanced: Multiple Joins)

```
1  Allocation and transfer history
2  SELECT s.student_name,h.hostel_name,r.room_no,b.bed_no,a.start_date,a.end_date
3  FROM allocation a JOIN student s ON s.student_id=a.student_id
4  JOIN bed b ON b.bed_id=a.bed_id JOIN room r ON r.room_id=b.room_id
5  JOIN hostel h ON h.hostel_id=r.hostel_id ORDER BY s.student_id,a.start_date;
6
```

Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.012 seconds

	STUDENT_NAME	HOSTEL_NAME	ROOM_NO	BED_NO	START_DATE	END_DATE
1	Student 01	H01 - Kaveri	2	1	8/1/2026, 12:00:00	9/1/2026, 12:00:00
2	Student 01	H01 - Kaveri	1	1	9/1/2026, 12:00:00	*(null)*
3	Student 02	H01 - Kaveri	1	2	9/1/2026, 12:00:00	*(null)*
4	Student 03	H01 - Kaveri	2	1	9/1/2026, 12:00:00	*(null)*
5	Student 04	H01 - Kaveri	2	2	9/1/2026, 12:00:00	*(null)*
6	Student 05	H01 - Kaveri	3	1	9/1/2026, 12:00:00	*(null)*
7	Student 06	H01 - Kaveri	3	2	9/1/2026, 12:00:00	*(null)*
8	Student 07	H02 - Krishna	1	1	9/1/2026, 12:00:00	*(null)*
9	Student 08	H02 - Krishna	1	2	9/1/2026, 12:00:00	*(null)*
10	Student 09	H02 - Krishna	1	3	9/1/2026, 12:00:00	*(null)*

This query retrieves students' accommodation history, including hostel, room, bed, allocation start date, and allocation end date. It joins allocation records with student and accommodation tables to present a historical record of residence assignments and transfers.

7. Results and Discussion

7.1 Results

The University Hostel and Accommodation Management System was designed and implemented using Oracle SQL. The database integrates student information, hostel facilities, accommodation allocations, fee transactions, attendance, maintenance complaints, visitor records, and disciplinary incidents.

7.2.1 Student Support Risk Classification and Decision Support

The Student Support Risk Identification module integrates attendance records, outstanding hostel fees, unresolved maintenance complaints, and disciplinary history to identify current residents who may require administrative support. The system uses the V_STUDENT_SUPPORT_STATS and V_STUDENT_SUPPORT_RISK views to consolidate relevant information, calculate risk scores, and classify students into three categories: HIGH PRIORITY, NEEDS ATTENTION, and NORMAL.

Each of the four risk indicators contributes between zero and two points, resulting in a total score ranging from zero to eight. Students scoring five or more are classified as HIGH PRIORITY, those scoring between two and four are classified as NEEDS ATTENTION, and those scoring zero or one are classified as NORMAL, provided their attendance data is complete. If attendance records are missing or the attendance percentage cannot be determined, the student is classified as at least NEEDS ATTENTION, even when the calculated score is low.

The model also considers data quality to prevent incomplete attendance records from being interpreted as evidence of normal status. The resulting classification supports more focused administrative review by highlighting cases that may require attendance follow-up, fee assistance,

maintenance resolution, or further counselling and review. The criteria are illustrative design choices rather than official university policies or a statistically validated prediction model.
The integrated report generated by Query 10 presents student-level indicators, risk scores, data-quality information, and risk classifications. Query 21 further generates a prioritized administrative attention queue containing current residents whose risk status is not NORMAL. This allows administrators to review higher-priority cases first and determine appropriate follow-up actions based on the underlying evidence.
Overall, the risk classification module demonstrates how relational data and analytical SQL views can support explainable, data-driven hostel administration. The classification is intended to guide support and review, not to establish misconduct or automatically impose disciplinary action.

7.2.2 Discussion

The implementation demonstrates how a relational database can improve the organization and retrieval of hostel management information.
• Data organization and integrity: The relational structure separates information into related tables, reducing unnecessary duplication. Primary keys, foreign keys, unique constraints, check constraints, and triggers help maintain valid records and enforce business rules.
• Accommodation management: The occupancy and room utilization queries provide information about occupied beds, available beds, and hostel capacity. The allocation validation rules prevent overlapping allocations for the same student or bed and restrict allocation of out-of-service beds.
• Financial management: The fee balance view and financial queries calculate charges, payments, outstanding amounts, and overdue fees. The expected outstanding balance of ₹47,000 provides an aggregate figure for monitoring unpaid hostel fees.
• Attendance and student monitoring: Attendance queries distinguish present days, absent days, approved leave, and missing attendance records. These indicators help administrators identify attendance patterns and students who may require follow-up.
• Maintenance and disciplinary analysis: Complaint and expense queries help identify recurring maintenance categories and compare hostel maintenance expenditure. Disciplinary queries summarize incidents by hostel and severity, supporting administrative review.
• Decision-support functionality: The student support risk view integrates attendance, outstanding fees, maintenance complaints, and disciplinary history. The resulting risk score and risk classification help prioritize administrative attention. The system also includes data-quality indicators to support interpretation of the results.
• Advanced SQL capabilities: Multi-table joins, aggregate functions, subqueries, a correlated subquery, and reusable views demonstrate the use of SQL for more complex hostel management requirements.
Overall, the implementation provides a structured approach to hostel administration and supports reporting across multiple operational areas.

8. Conclusion

The University Hostel and Accommodation Management System was developed as a relational database solution for managing hostel-related information using Oracle SQL. It integrates student records, hostel facilities, room and bed allocations, fee management, attendance, maintenance complaints, visitors, and disciplinary records.
The project demonstrates the use of relational schema design, primary and foreign keys, integrity constraints, indexes, triggers, data manipulation operations, views, and analytical SQL queries. These features support data consistency, accurate accommodation allocation, financial monitoring, and efficient retrieval of administrative information.
The analytical queries and student support risk identification module demonstrate how data from multiple tables can be combined to generate useful administrative insights. Occupancy statistics, outstanding fee balances, attendance indicators, maintenance reports, and disciplinary summaries can assist hostel administrators in monitoring operations and identifying cases requiring intervention.

The modular implementation, consisting of separate schema, sample data, view, verification, and reporting scripts, also improves maintainability and makes the system easier to test and extend.
In conclusion, the project demonstrates the practical application of database management concepts to a university hostel environment. Future improvements could include a graphical user interface, role-based access control, automated fee notifications, online complaint submission, and real-time hostel availability monitoring.